



DSA Assignment - 3

Unit–3 Assignment: Sorting Performance Analyzer (SPA)

Course Details

Name of the School: School of Engineering & Technology

Student Name: Tanuj

Section: A

Program (Class / Semester / Batch): B.Tech CSE (AI MI) / 2 /
2025-29

Course Title: Data Structures

Course Code: ETCCDS202

Unit Title: Sorting Algorithms

Faculty: Neetu Chauhan

Submission Date: 17/04/2026

1. Execution time table

Input type	Size	Insertion Sort (ms)	Merge Sort (ms)	Quick Sort (ms)
random	1000	49.082	3.927	2.373
sorted	1000	0.464	4.286	94.247
reverse	1000	81.984	3.202	75.897
random	5000	1333.766	39.944	13.446
sorted	5000	2.642	29.593	2454.653
reverse	5000	2598.479	17.118	1861.911
random	10000	4928.144	62.367	33.326
sorted	10000	3.198	40.132	10708.624
reverse	10000	11875.628	89.870	9829.765

These 27 measurements cover 3 input sizes × 3 input types × 3 algorithms.

2. Comparison by input type

Random inputs

For random data, insertion sort is much slower than merge sort and quick sort, especially as the input size grows. For example, at size 10000, insertion sort takes about 4928 ms, while merge sort is around 62 ms and quick sort about 33 ms. This matches the expectation that an $O(n^2)$ algorithm scales poorly compared to $O(n \log n)$ algorithms on large, random inputs.

Sorted inputs

On already sorted data, insertion sort becomes extremely fast, staying below about 4 ms even for 10000 elements. Merge sort remains consistently in the few-tens of milliseconds because it always does the same divide-and-conquer work regardless of input order. Quick sort with last-element pivot performs very poorly on sorted input, reaching over 10 seconds at size 10000, which reflects its worst-case behavior.

Reverse-sorted inputs

On reverse-sorted data, insertion sort is slower than on sorted data and shows clear quadratic growth, reaching more than 11 seconds at size 10000. Merge sort again stays relatively stable and fast (under 100 ms even for 10000 elements). Quick sort is also very slow on reverse-sorted input, with times close to its worst case (about 9.8 seconds at size 10000).

3. Big-O vs timings ($O(n^2)$ vs $O(n \log n)$)

Insertion sort has time complexity $O(n^2)$, which means its running time grows roughly with the square of the input size. This is visible in the results shown in the above table: for random input, increasing the size from 1000 to 10000 makes the time jump from about 49 ms to about 4928 ms, roughly 100× increase for a 10× increase in n .

Merge sort and quick sort both have average-case time complexity $O(n \log n)$, so their running time grows much more slowly. In the random input data, merge sort goes from about 4 ms to about 62 ms, and quick sort from about 2 ms to about 33 ms when the size increases from 1000 to 10000, which is much closer to linear-times-logarithmic growth. Overall, the experimental results clearly support the theoretical difference between $O(n^2)$ and $O(n \log n)$.

4. Special note on Quick Sort

Quick sort's worst-case time complexity is $O(n^2)$, and this happens when the pivot choice creates very unbalanced partitions (for example, always picking the smallest or largest element as pivot). In my implementation, the pivot is the last element, so already sorted or reverse-sorted arrays produce highly unbalanced partitions and trigger the worst case.

The timings show this clearly: for sorted inputs, quick sort times explode from about 94 ms at 1000 elements to over 10708 ms (about 10.7 seconds) at 10000 elements. Reverse-sorted inputs show a similar pattern, with quick sort reaching about 9829 ms at size 10000, which is again consistent with worst-case $O(n^2)$ behavior.

5. Stability and memory usage

- Insertion sort is stable and in-place: it keeps equal elements in their original relative order and uses only constant extra memory.
- Merge sort is stable but out-of-place in its standard form, because it uses extra arrays to merge the sublists, so it needs additional memory proportional to n .
- Quick sort is usually in-place (it rearranges elements inside the same array with only a small stack/recursion overhead) but is not stable in typical implementations because swapping can change the relative order of equal elements.

These properties explain why merge sort often has a slightly higher memory cost but predictable performance, while quick sort is memory-efficient and fast on average but can be unstable and very slow in its worst case.